

ARE 212 SECTION 1:

Introduction to R

Fiona Burlig

January 21, 2016

In this section, I'll go over the syllabus, introduce you to R with a brief example using real data (!), and make a plug for `knitr`. If there's time left over, we'll go through some linear algebra puzzles that will utilize your newfound R skills.¹ If you're an experienced R user, things might feel a little slow at the beginning, but I'll be interspersing course material into the exercises as we go, and we'll speed up soon.

Installing R

The first thing you have to do to use R is to download it from here:

<http://cran.r-project.org>

The online documentation and installation routines are comprehensive. Simply choose the version built for your operating system, and away you go. I highly recommend also using `Rstudio`, a popular integrated development environment (IDE) that has a variety of nice features. You can download it here:

<https://www.rstudio.com/products/rstudio/download/>

If you're already down the Linux rabbit hole, Emacs with the Emacs Speaks Statistics (ESS) package is a popular way of running R. If you're not already a Linux user, carefully consider the MB/MC involved in switching.²

Some R basics

One of R's best features is that it's based on packages. R by itself is pretty good, but R plus packages is incredibly powerful. Nearly any time you do anything in R, you'll be using functions or datasets contained within a package. The first time you use a package, you'll have to download and install it first. Thankfully, R makes this super easy. Let's give it a try with the `haven` package:³

```
install.packages("haven")
```

We're actually going to install one more package - `dplyr` - which will make our lives way easier later (trust me on this one)⁴:

¹This is unlikely. I strongly encourage you to play around with these on your own before the next section.

²I personally use R with `Rstudio` on both a Mac and a PC. I also personally use Stata. And Python. And ArcGIS. And Matlab. My exclusive-R-user-friends give me a hard time about this.

³As we're about to see, `haven` lets R read in datasets from SAS, SPSS, and Stata. It supports Stata 13 and 14, which makes it better than `foreign`. It also reads things in as `tbl_df` objects, which will come in handy later on. But: it's also still in the somewhat early stages of development, so *caveat econometricus* (let the econometrician beware).

⁴Okay, fine. `dplyr` is a data manipulation package. We're loading it now because `haven` and `dplyr` play nicely together. As we'll learn, R has a bunch of ways of storing data. The `tbl_df` format is my personal favorite. Loading `dplyr` enables the use of this format.

```
install.packages("dplyr")
```

Once you've installed a package (which you only have to do once), we need to tell R to hold its contents in memory for our current program. We do this with the `library()` function:

```
library("haven")  
library("dplyr")
```

Now we can start working with some actual data! I've put the classic Stata `autos.dta` dataset on bCourses and on my website; if you haven't already, download it to your ARE 212 section folder. Before we open the dataset, a quick pro tip:

Pro tip: Everything you do in R should be contained in a `.R` script file for the purposes of reproducibility. Good coding practice is to begin your `.R` files by loading all the packages you'll need for that script at the top. Another good (read: essential) practice is to tell R where to find the datasets it will need to run your script. We do this by setting the working directory (shown below). Set the working directory *once* at the top of your script, and then use relative paths in the script that follows. Like this:

```
#this is an absolute path:  
# note the '/Users' at the beginning.  
# PC equivalent: 'C:/' at the beginning of the path  
setwd("/Users/fwilkes1/Desktop/Dropbox/UC Berkeley/2015-2016/ARE 212/")  
# You can also check your current working directory with the getwd() function:  
getwd()  
  
## [1] "/Users/fwilkes1/Desktop/Dropbox/UC Berkeley/2015-2016/ARE 212"
```

Now we're in the right place, let's get started.

Chasing `[autos.dta]`

To read the Stata dataset `autos.dta` into R, let's use one of the functions in our newly-installed `haven` package, conveniently titled `read_dta()`:

```
#note the relative path here.  
# My WD is already set to "ARE 212", but my file is in "ARE 212/Section 1".  
# Rather than set the wd again, I just tell R where to find my file.  
  
# The "<-" operator assigns the stuff on the right to a new object  
# It's called the stuff on the left.  
cars_data <- read_dta("Section 1/autos.dta")
```

Let's take a look at the output.

```
cars_data  
  
## Source: local data frame [74 x 12]
```

```
##
##           make price   mpg rep78 headroom trunk weight length
##           (chr) (int) (int) (int)   (dbl) (int)  (int)  (int)
## 1    AMC Concord  4099    22     3     2.5    11   2930   186
## 2    AMC Pacer   4749    17     3     3.0    11   3350   173
## 3    AMC Spirit  3799    22    NA     3.0    12   2640   168
## 4  Buick Century  4816    20     3     4.5    16   3250   196
## 5  Buick Electra  7827    15     4     4.0    20   4080   222
## 6  Buick LeSabre 5788    18     3     4.0    21   3670   218
## 7    Buick Opel  4453    26    NA     3.0    10   2230   170
## 8    Buick Regal  5189    20     3     2.0    16   3280   200
## 9  Buick Riviera 10372    16     3     3.5    17   3880   207
## 10 Buick Skylark  4082    19     3     3.5    13   3400   200
## ..           ...    ...    ...    ...    ...    ...    ...
## Variables not shown: turn (int), displacement (int), gear_ratio
##           (dbl), foreign (lbl)
```

That's a nice looking dataset. Things of note here: we see at the top of the output that this is a "local data frame" object (aka `tbl_df`), with 72 rows and 12 columns. We can see the type of data stored in each column below the column title. And R is smart enough not to output the entire dataset - this is manageable.⁵ (If you do want to bring up the whole thing, and you're using `Rstudio`, type `View(cars_data)` to bring up a new window with the whole data table.)⁶

This dataset is a little big, though. We don't need all of it - let's only keep a couple of columns. This is super easy using `dplyr`:

```
# Wait, what was in this dataset again? Let's check.
names(cars_data)

## [1] "make"      "price"     "mpg"       "rep78"
## [5] "headroom"  "trunk"     "weight"    "length"
## [9] "turn"      "displacement" "gear_ratio" "foreign"

# Okay. Let's keep price, mpg, and weight for now.

# Note: the first entry is the name of the dataset we're starting with
# The second, third, and fourth entries are variable names.
# We name our new output cars_data_small.

cars_data_small <- select(cars_data, price, mpg, weight)

# And take a look:

cars_data_small
```

⁵If your output doesn't look like this, chances are that you forgot to run `library(dplyr)`. Try again. Do not pass Go. Do not collect \$200.

⁶You can also get R to print the entire dataset into the console with the `print()` command, where you can specify a number of rows as follows: `print(cars_data, n=DESIRED NR OF ROWS)`, or with the `head()` function as we'll see later. R is full of multiple ways to do the same thing!

```
## Source: local data frame [74 x 3]
##
##      price    mpg weight
##      (int) (int)  (int)
## 1    4099     22   2930
## 2    4749     17   3350
## 3    3799     22   2640
## 4    4816     20   3250
## 5    7827     15   4080
## 6    5788     18   3670
## 7    4453     26   2230
## 8    5189     20   3280
## 9   10372     16   3880
## 10   4082     19   3400
## ..      ...      ...      ...
```

Nice. Just as easy as in Stata. I'm starting to like this `dplyr` thing.

A brief detour

“But what if my data aren’t Stata, SPSS or SAS?” you ask. Easy. R has a litany of built-in functions to read other data types, like CSVs, and the `foreign` package does basically everything else, including older Stata files. We’ll quickly use R’s built-in function to read in a CSV:

```
# The syntax is very similar, but not identical (note "." rather than "-")
cars_csv <- read.csv("Section 1/autos.csv", header = TRUE)
```

We do have to do one additional step: covert our new `cars_csv` object into a `tbl_df` object. There’s no reason to leave things in the default format - `tbl_df` does everything it can do and more.

```
cars_tbl_df <- tbl_df(cars_csv)
```

And just look to make sure it looks good:

```
cars_tbl_df

## Source: local data frame [74 x 3]
##
##      V1     V2    V3
##      (int) (int) (int)
## 1    4099     22  2930
## 2    4749     17  3350
## 3    3799     22  2640
## 4    4816     20  3250
## 5    7827     15  4080
## 6    5788     18  3670
## 7    4453     26  2230
## 8    5189     20  3280
```

```
## 9  10372    16  3880
## 10  4082    19  3400
## ..    ...    ...    ...
```

Hey! Some jerk gave us unlabeled data!⁷ Let's fix that:⁸

```
names(cars_tbl_df) <- c("price", "mpg", "weight")
```

Much better. Note, though, that now we have two identical datasets in memory. Don't believe me? Good - I can prove it. Let's ask R: is every element in `cars_data_small` the same as in `cars_tbl_df`?⁹

```
# We could write (without the "#", of course):
### cars_data_small == cars_tbl_df

# == denotes equality; we're asking element by element bc these things are data frames.
```

But that would be an annoying output (try it yourself and see). Let's only show the top 6 rows:

```
head(cars_data_small == cars_tbl_df)

##      price  mpg weight
## [1,]  TRUE  TRUE   TRUE
## [2,]  TRUE  TRUE   TRUE
## [3,]  TRUE  TRUE   TRUE
## [4,]  TRUE  TRUE   TRUE
## [5,]  TRUE  TRUE   TRUE
## [6,]  TRUE  TRUE   TRUE
```

Since you're now convinced, let's get rid of `cars_tbl_df`. What an annoying name:

```
rm("cars_tbl_df")
```

On the road again

Let's get back to it. First, let's take a look at the data one more time. I want to see the first 8 observations, so I can write:

```
head(cars_data_small, 8)

## Source: local data frame [8 x 3]
##
##   price    mpg weight
##   (int) (int)  (int)
## 1  4099     22   2930
```

⁷I downloaded this dataset from <https://github.com/pbaylis/ARE212/blob/master/section-01/>.

⁸You can also do this with the `dplyr` command `rename()`.

⁹You can also do this with the `all.equal()` command - probably more useful in real life.

```
## 2  4749    17   3350
## 3  3799    22   2640
## 4  4816    20   3250
## 5  7827    15   4080
## 6  5788    18   3670
## 7  4453    26   2230
## 8  5189    20   3280
```

Now I want to see just the price column. Since the columns are named, we can use the `$` operator to request the price variable from `cars_data_small`:

```
head(cars_data_small$price, 8)

## [1] 4099 4749 3799 4816 7827 5788 4453 5189
```

Whoa! Cars are cheap (or are they? What are the units???) Let's look at something else. I own an awesome 2004 Subaru with a sweet turbo engine - but it only gets about 21 MPG.¹⁰ How does that stack up against the cars in this dataset? Let's take a look. The first thing we want to do is sort the dataset by fuel economy. Again, `dplyr` makes this easy:

```
# Again, the first entry is the data name, and the second the sorting variable
cars_sorted <- arrange(cars_data_small, mpg)

cars_sorted

## Source: local data frame [74 x 3]
##
##   price    mpg weight
##   (int) (int)  (int)
## 1  11497     12   4840
## 2  13594     12   4720
## 3  11385     14   4330
## 4  14500     14   3900
## 5  13466     14   3830
## 6   5379     14   4060
## 7   6303     14   4130
## 8  12990     14   3420
## 9   7827     15   4080
## 10  6165     15   3720
## ..     ...     ...
```

Okay, encouraging (for me, not the environment): the worst performers in this dataset are much worse than me. But what about the best? Use the `desc(ending)` qualifier in the `arrange` command to look:

```
# Again, the first entry is the data name, and the second the sorting variable
cars_sorted_desc <- arrange(cars_data_small, desc(mpg))

cars_sorted_desc
```

¹⁰Environmental economist shame moment.

```
## Source: local data frame [74 x 3]
##
##   price    mpg weight
##   (int) (int)  (int)
## 1   5397    41   2040
## 2   4589    35   2020
## 3   3798    35   2050
## 4   4425    34   1800
## 5   3748    31   2200
## 6   3984    30   2120
## 7   3995    30   1980
## 8   3299    29   2110
## 9   4389    28   1800
## 10  4647    28   3260
## .. ... ..
```

Okay, significantly better than me. What about the average?

```
# First input is data, second is the stat we want:
average_mpg <- summarize(cars_data_small, mean(mpg))

average_mpg

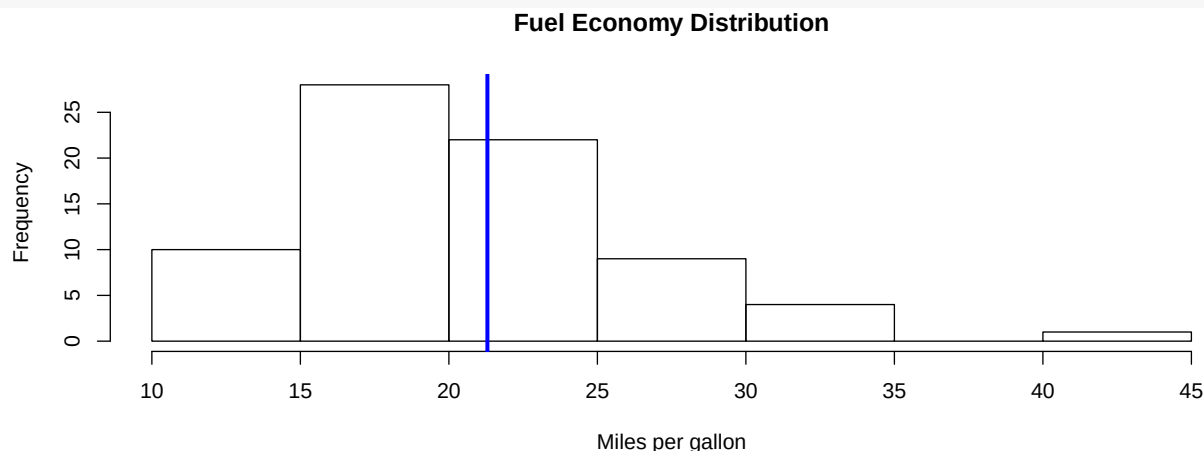
## Source: local data frame [1 x 1]
##
##   mean(mpg)
##   (dbl)
## 1   21.2973
```

I've never been so excited to be average in my life! But of course, one summary statistic is for suckers. Let's actually look at our data.

Disclaimer: In a later section, I'll introduce you to `ggplot2`, the reason that people really love R graphics. We don't have time to do that today, but don't put too much energy into learning the basic plot functions.

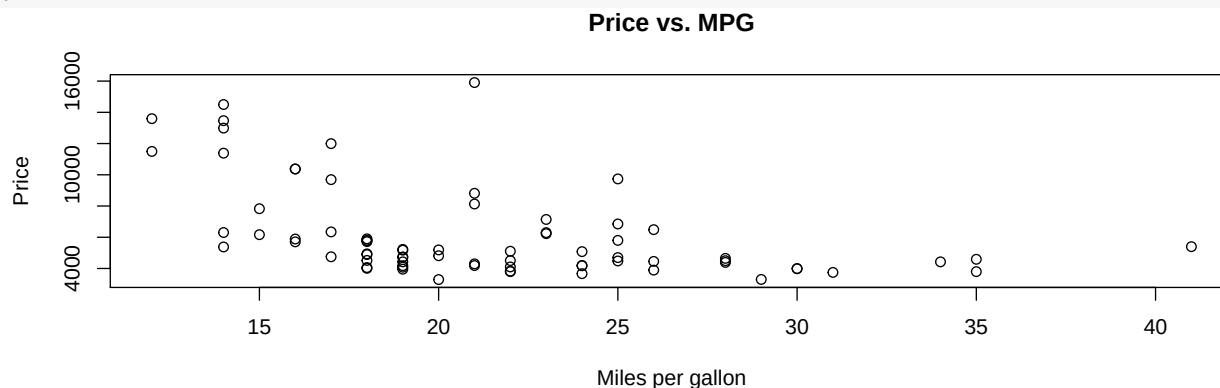
First, let's close the loop on my Subaru's MPG by looking at the distribution of fuel economy in our data, and add a vertical line at the mean:

```
# Call hist on the data we're interested in (only one variable here)
# Title the figure with main = ""
# Title the x axis with xlab = ""
# Y axis already defaults to what we want.
hist(cars_data_small$price,
     main="Fuel Economy Distribution",
     xlab = "Miles per gallon")
# Now add a line of value average_mpg, color blue, and line width 3.
abline(v=average_mpg,col="blue", lwd = 3)
```



Finally, let's see if there's a correlation between MPG and price using a simple scatterplot:

```
# Call plot on the data we're interested in (2 variables here: X then Y)
plot(cars_data_small$mpg, cars_data_small$price,
     main="Price vs. MPG",
     xlab = "Miles per gallon",
     ylab = "Price")
```



There seems to be a negative correlation here: the higher the MPG, the lower the price. Maybe this is the SUV effect?

A plug for knitr

You should use `knitr` for this class! It integrates `R` code and `LATEX` really nicely. My section notes will be written in `knitr`, so feel free to use this document as a template. I'll take you through `knitr` next week, but by all means get a head start if you can and want. Before next section, please make sure you have a `LATEX` distribution and a PDF reader installed. You can get your `LATEX` distribution from <http://latex-project.org/ftp.html>.¹¹

Linear algebra puzzles

Try your hand at the puzzles below - they provide a code illustration of the linear algebra review in Chapter 1 of the lecture notes. Don't worry if you can't solve them just yet - come back to them later, once we have gone over `R` code in more detail. There are many correct ways to solve these puzzles. If there's time, we might look at them in section.

1. Let $\mathbf{I}_5$ be a 5×5 identity matrix. Demonstrate that $\mathbf{I}_5$ is symmetric and idempotent using simple functions in `R`.
2. Generate a 2×2 idempotent matrix $\mathbf{X}$, where $\mathbf{X}$ is not the identity matrix. Demonstrate that $\mathbf{X} = \mathbf{X}\mathbf{X}$.
3. Generate two random variables, $\mathbf{x}$ and $\mathbf{e}$, of dimension $n = 100$ such that $\mathbf{x}, \mathbf{e} \sim N(0, 1)$. Generate a random variable $\mathbf{y}$ according to the data generating process $y_i = x_i + e_i$. Show that if you regress $\mathbf{y}$ on $\mathbf{x}$ using the canned linear regression routine `lm()`, then you will get an estimate of the intercept β_0 and the coefficient on $\mathbf{x}$, β_1 , such that $\beta_0 = 0$ and $\beta_1 = 1$.
4. Show that if $\lambda_1, \lambda_2, \dots, \lambda_5$ are the eigenvalues of a 5×5 matrix $\mathbf{A}$, then $\text{tr}(\mathbf{A}) = \sum_{i=1}^5 \lambda_i$.

¹¹The Mac's built-in PDF reader is just fine. If you're a PC user, go grab Sumatra from <http://www.sumatrapdfreader.org/free-pdf-reader.html>.